Task 1: Data Design Review (6 Marks)

Task 1.1: Diagnose the Design (2 Marks)

The original design has eight deficiencies. Each is a design problem rather than a syntax problem.

#	Deficiency	What could go wrong in the health centre
1	Parallel arrays: one patient is spread over name[], age[], phone[], bp[] and visitDate[], tied together only by the index.	If one array is sorted, shifted or deleted from without the others, a patient's name gets paired with another patient's BP or phone number. The wrong patient is called, or the wrong BP history is shown to the doctor.
2	A 10-digit phone number is stored in char phone[100][10].	A string of 10 digits needs 11 bytes (the extra one is the terminating '\0'). strcpy writes that 11th byte into the first byte of phone[i+1], because rows are contiguous. The next patient's number is silently damaged, so reminder calls fail. The text is also never validated (letters, wrong length).
3	Blood pressure such as 120/80 is stored in one float.	A float cannot hold two numbers. 120.80, 120.8 and "120/8" cannot be told apart, so systolic and diastolic values cannot be recovered. The hospital cannot test "systolic >= 140 or diastolic >= 90", so hypertension may be missed. Floats are also inexact in binary (120.8 is not stored exactly).
4	The visit date is stored as text in char visitDate[100][8].	"12/11/2026" needs 11 bytes and even "12/11/26" needs 9, so the date overflows into the next row. Text dates cannot be compared or subtracted (for example, to check a 14-day follow-up). Formats are ambiguous (dd/mm or mm/dd) and invalid dates such as 31/02 are accepted.
5	No limit check on count.	The 101st patient is written beyond the arrays, which is undefined behaviour. It can overwrite neighbouring globals (perhaps count itself), crash the program on a busy day, or corrupt earlier records.
6	Everything is a global variable.	Any function can change the arrays or count by accident. A second register (a separate OPD and antenatal clinic, or a test copy) is impossible. The code cannot be unit tested and it is not thread-safe.
7	Sorting by age is hard.	Five arrays must be swapped in lockstep. Forgetting even one swap silently corrupts the whole register. Adding a field such as gender or blood group means changing every swap and every function. Listing older patients first for triage becomes error-prone.
8	addPatient takes five loose parameters and does no checking. Also, name[100][20] holds only 19 characters.	Arguments can be passed in the wrong order (for example, age and BP) and it still compiles. Impossible values (age -5, BP 0) are accepted. A 25-character name overflows into the next record through unchecked strcpy.

Task 1.2: Repair the Design (3 Marks)

Each patient becomes one structure. The date and the blood pressure are nested structures, the phone buffer has room for the terminator, and the array and its count are bundled in one register structure that is passed by pointer. addPatient() now receives a structure pointer and checks capacity.

patient_register.c (compiled and tested)

```
#include <stdio.h>
#include <string.h>

#define MAX_PATIENTS 100
#define NAME_LEN     30
#define PHONE_LEN    11      /* 10 digits + '\0' */

struct date { int day, month, year; };
struct bp   { int systolic, diastolic; };

struct patient {
```

```
    char        name[NAME_LEN];
    int         age;
    char        phone[PHONE_LEN];
    struct bp   bp;
    struct date visit;
};

struct patient_register {
    struct patient list[MAX_PATIENTS];
    int count;
};

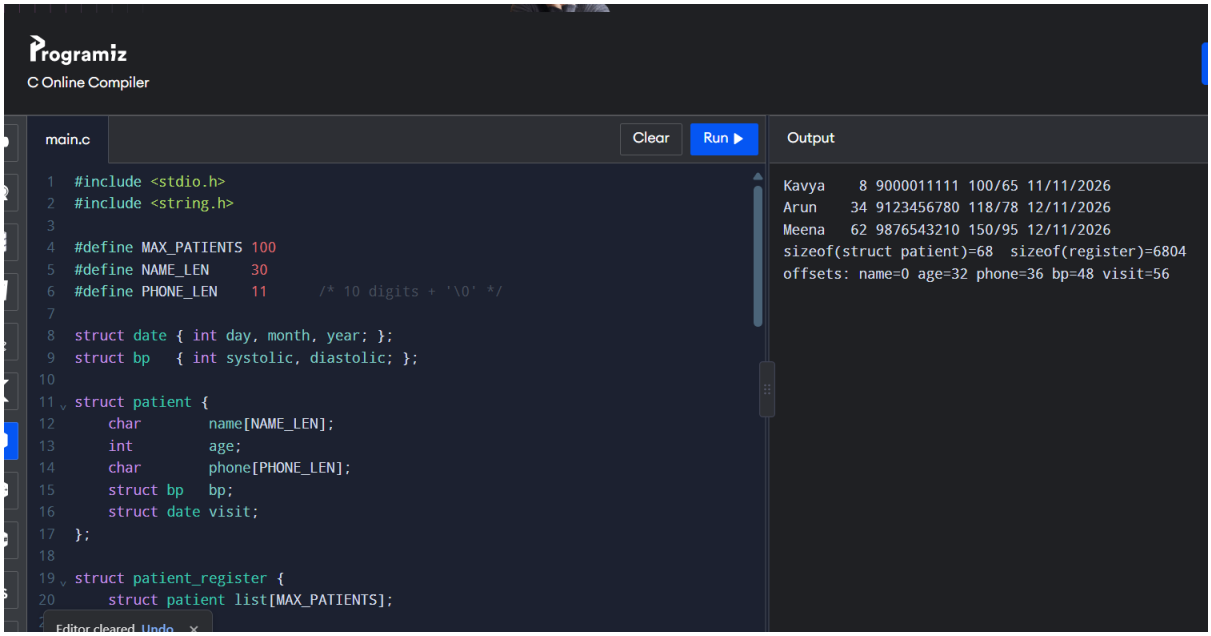
/* returns 1 on success, 0 if the register is full */
int addPatient(struct patient_register *reg, const struct patient *p)
{
    if (reg == NULL || p == NULL || reg->count >= MAX_PATIENTS)
        return 0;
    reg->list[reg->count] = *p;       /* one structure assignment */
    reg->count++;
    return 1;
}

void sortByAge(struct patient_register *reg)
{
    for (int i = 0; i < reg->count - 1; i++) {
        int min = i;
        for (int j = i + 1; j < reg->count; j++)
            if (reg->list[j].age < reg->list[min].age) min = j;
        if (min != i) {
            struct patient t = reg->list[i];
            reg->list[i] = reg->list[min];
            reg->list[min] = t;
        }
    }
}

int main(void)
{
    static struct patient_register reg;          /* count starts at 0 */
    struct patient a = {"Meena", 62, "9876543210", {150, 95}, {12, 11, 2026}};
    struct patient b = {"Arun",  34, "9123456780", {118, 78}, {12, 11, 2026}};
    struct patient c = {"Kavya",  8, "9000011111", {100, 65}, {11, 11, 2026}};
    addPatient(&reg, &a); addPatient(&reg, &b); addPatient(&reg, &c);
    sortByAge(&reg);
    for (int i = 0; i < reg.count; i++)
        printf("%-6s %3d %s %d/%d %02d/%02d/%d\n", reg.list[i].name, reg.list[i].age,
               reg.list[i].phone, reg.list[i].bp.systolic, reg.list[i].bp.diastolic,
               reg.list[i].visit.day, reg.list[i].visit.month, reg.list[i].visit.year);
    printf("sizeof(struct patient)=%zu  sizeof(register)=%zu\n",
           sizeof(struct patient), sizeof(struct patient_register));
    printf("offsets: name=%zu age=%zu phone=%zu bp=%zu visit=%zu\n",
        __builtin_offsetof(struct patient,name), __builtin_offsetof(struct patient,age),
        __builtin_offsetof(struct patient,phone), __builtin_offsetof(struct patient,bp),
        __builtin_offsetof(struct patient,visit));
    return 0;
}
```

Output

```
Kavya    8 9000011111 100/65 11/11/2026
Arun    34 9123456780 118/78 12/11/2026
Meena   62 9876543210 150/95 12/11/2026
sizeof(struct patient)=68  sizeof(register)=6804
offsets: name=0 age=32 phone=36 bp=48 visit=56
```



Changes and why:

- phone[11] holds 10 digits plus '\0'; struct bp keeps systolic and diastolic as separate integers; struct date stores day, month and year as integers, so dates can be compared and subtracted.
- MAX_PATIENTS is checked inside addPatient(), which returns 0 when the register is full. The caller must handle that result.
- No globals. The register is created by the caller and passed as struct patient_register *. Two registers can now coexist.
- reg->list[reg->count] = *p; copies the whole record with one structure assignment. This replaces five strcpy or assignment statements and cannot misalign fields. The input p is const because it is only read.

Task 1.3: Design Defence (1 Mark, 85 words)

Each patient is one self-contained structure, so sorting swaps whole records with a single assignment and the fields can never become mismatched. A pointer passes the entire record, or the whole register, to a function as one argument, avoiding long parameter lists. To extend the design, add a field such as gender to struct patient; existing code keeps working. The nested date and blood-pressure types are reusable and comparable, and the register bundles the array with its count, so capacity checks travel with the data.

Alternative Concept and Complexity Analysis

When are parallel arrays (structure of arrays, SoA) still preferred?

- When most operations scan **one field** across many records (for example, the average age or "count patients older than 60"). The age[] array is contiguous (4-byte stride), whereas in an array of structures the ages are 68 bytes apart, so far fewer cache lines are loaded.
- When the compiler or hardware can vectorise (SIMD or GPU) a single-field loop, and in column-store databases and analytics.
- When each field is stored on its own, for example only one column must be written to disk, or fields are added and dropped independently of each other.
- Padding is avoided: arrays of one type are packed with no gaps between elements.

Sorting n patients by age (selection sort, at most n-1 swaps). Both designs make n(n-1)/2 comparisons, so the time is O(n^2) in both (O(n log n) with merge sort or qsort). They differ in the cost of one swap:

	Array of structures (AoS)	**Parallel arrays (SoA)**
Elements moved per swap	1 element: the whole structure (68 bytes)	5 elements, one in each array (20+4+10+4+8 = 46 bytes in the original sizes)
Copy work per swap (temp-based)	3 x 68 = 204 bytes, in 3 statements	3 x 46 = 138 bytes, in 15 statements (5 arrays x 3)
Worst-case bytes copied while sorting	up to 204(n-1) bytes	up to 138(n-1) bytes
Comparison phase	Stride of 68 bytes between ages: poorer cache use	Stride of 4 bytes: better cache use
Risk	None (the record stays together)	High: a missed array swap corrupts the data
Space (corrected field sizes)	100 x 68 + 4 = 6804 bytes (measured). 65 useful bytes per record + 3 bytes of padding (2 after name, 1 after phone) = 300 bytes of padding, about 4.4%.	100 x (30+4+11+8+12) + 4 = 6504 bytes with no padding between records

The flawed original uses only 4604 bytes (measured: 2000+400+1000+400+800+4), but only because its fields are too small, which is exactly the overflow problem diagnosed above. For either design, sorting an array of **indices or pointers** (4 to 8 bytes per swap) removes most of the copying cost.